

Custom Hooks

- Custom Hooks in React are a powerful feature that allows us to extract component logic into reusable functions.
- These Hooks are JavaScript functions that can use other Hooks provided by React. They enable us to organize logic into separate, reusable modules.

Why Use Custom Hooks?

- Encapsulation: Keeps component logic separate and reusable.
- Reusability: Use the same hook across different components.
- Code Organization: Reduces clutter in components.
- Improved Readability: Complex logic is abstracted into a single function.

Tips for Building Custom Hooks

- Start by identifying repetitive logic across your components.
- Extract this logic into a function named with use prefix. Custom hooks should start with use, like useFormInput or useFetch.
- Use React's built-in Hooks within your custom Hook as needed.
- Return anything that will be useful for the component using this Hook.
- Each custom hook should be responsible for a single piece of functionality.

Example 1: useCounter Hook

We'll build a useCounter hook that encapsulates the logic for incrementing a counter.

```
import { useState } from 'react'

function useCounter(initialValue = 0) {
  const [count, setCount] = useState(initialValue) // State to keep track of count

  const increment = () => setCount(prevState => prevState + 1)

  return [count, increment]
}

export default useCounter
```

To use our useCounter Hook, we can include it in a functional component

```
import React from 'react';
import useCounter from './useCounter'

function CounterComponent() {
  const [count, increment] = useCounter()

  return (
    <div>
      <p>Count: {count}</p>
      <button onClick={increment}>Increment</button>

    </div>
  )
}
export default CounterComponent
```

Example 2: useFetch

If we need to fetch data from an API in several components, we can create a useFetch Hook.

```
import { useState, useEffect } from "react";

const useFetch = (url) => {
  const [data, setData] = useState(null);
```

```

const [loading, setLoading] = useState(true);
const [error, setError] = useState(null);

useEffect(() => {
  fetch(url)
    .then((response) => {
      if (!response.ok) throw new Error("Failed to fetch data");
      return response.json();
    })
    .then((result) => setData(result))
    .catch((err) => setError(err.message))
    .finally(() => setLoading(false));
}, [url]);

return { data, loading, error };
};

export default useFetch;

```

We can now use this Hook in our components to fetch data easily:

```

import React from "react";
import useFetch from "../useFetch";

const UsersList = () => {
  const { data, loading, error } = useFetch("https://jsonplaceholder.typicode.com/users");

  if (loading) return <p>Loading...</p>;
  if (error) return <p>Error: {error}</p>;

```

```

return (
  <div>
    <h1>Users List</h1>
    <ul>
      {data.map((user) => (
        <li key={user.id}>{user.name}</li>
      ))}
    </ul>
  </div>
);
};

export default UsersList;

```

Example 3: useToggle

This hook toggles a boolean value (useful for modals, dark mode, etc.).

```

import { useState } from "react";

const useToggle = (initialValue = false) => {
  const [value, setValue] = useState(initialValue);
  const toggle = () => setValue((prev) => !prev);
  return [value, toggle];
};

export default useToggle;

```

This is how you can use it in a component

```

import React from "react";
import useToggle from "../useToggle";

```

```
const ToggleComponent = () => {  
  const [isVisible, toggleVisibility] = useToggle();  
  
  return (  
    <div>  
      <button onClick={toggleVisibility}>  
        {isVisible ? "Hide" : "Show"} Text  
      </button>  
      {isVisible && <p>This is some text!</p>}  
    </div>  
  );  
};  
  
export default ToggleComponent;
```